

# TP 6

## Gestion des artefacts

Container registry, versioning sémantique  
et automatisation des releases

Usine Logicielle — Master 1 DevOps  
Sup de Vinci 2025/2026  
Dorian Voets

### Informations pratiques

|                    |                                                  |
|--------------------|--------------------------------------------------|
| <b>Prérequis :</b> | TP5 terminé, image Docker déployée sur Cloud Run |
| <b>Rendu :</b>     | Dépôt GitHub avec releases + compte-rendu        |
| <b>Travail :</b>   | Individuel                                       |

### Sommaire

- Partie 0 — État des lieux
- Partie 1 — Multi-tagging des images Docker
- Partie 2 — Versioning sémantique (SemVer)
- Partie 3 — Releases GitHub automatisées
- Partie 4 — Pipeline de release complet
- Partie 5 — Recherche autonome

# Introduction

Ce TP fait suite au TP5 dans lequel vous avez déployé votre application Flask sur Cloud Run. Vous allez maintenant structurer la gestion de vos artefacts : multi-tagging des images Docker, versioning sémantique (SemVer), et automatisation des releases GitHub.

À la fin de ce TP, votre pipeline créera automatiquement des releases versionnées avec un changelog généré à partir de vos Conventional Commits.

## Partie 0 — État des lieux

**Objectif :** Vérifier le pipeline du TP5 et s'assurer que le déploiement fonctionne.

### 0.1 — Vérifications

Avant de commencer, vérifiez que :

- Votre pipeline CI/CD du TP5 est vert
- L'image Docker est publiée sur Artifact Registry
- L'application est déployée et accessible sur Cloud Run
- Vos commits suivent les Conventional Commits (feat:, fix:, docs:...)

**Question 1 (compte-rendu) :** Qu'est-ce qu'un artefact dans le contexte d'une usine logicielle ?  
Donnez 3 exemples d'artefacts différents.

## Partie 1 — Multi-tagging des images Docker

**Objectif :** Tagger chaque image avec plusieurs tags pour la traçabilité et le rollback.

### 1.1 — Comprendre les tags actuels

Actuellement, votre pipeline du TP5 tague les images avec `github.sha` et `latest`. C'est un bon début, mais en entreprise on utilise aussi des tags sémantiques.

### 1.2 — Ajouter le multi-tagging au pipeline

Modifiez le step de build dans votre job deploy pour ajouter un tag court du SHA. Mettez aussi à jour le step de push pour pousser tous les tags :

```
- name: Build et tag de l'image
  run: |
    IMAGE=europe-west1-docker.pkg.dev/${{ secrets.GCP_PROJECT_ID }}/flask-re
    po/flask-app
    SHORT_SHA=$(echo ${{ github.sha }} | cut -c1-7)
    docker build \
      -t $IMAGE:${{ github.sha }} \
      -t $IMAGE:sha-$SHORT_SHA \
```

```
-t $IMAGE:latest \  
.  
  
- name: Push de l'image (tous les tags)  
  run: |  
    IMAGE=europe-west1-docker.pkg.dev/${{ secrets.GCP_PROJECT_ID }}/flask-repo/flask-app  
    SHORT_SHA=$(echo ${github.sha} | cut -c1-7)  
    docker push $IMAGE:${github.sha}  
    docker push $IMAGE:sha-$SHORT_SHA  
    docker push $IMAGE:latest
```

**Astuce :** Le SHA court (7 caractères) est plus lisible que le SHA complet (40 caractères). C'est le format affiché par `git log --oneline`.

```
git add .github/workflows/ci.yml  
git commit -m "ci: ajout multi-tagging des images Docker"  
git push origin main
```

### 1.3 — Vérifier les tags sur le registre

```
# Lister les tags de l'image sur Artifact Registry  
gcloud artifacts docker tags list \  
  europe-west1-docker.pkg.dev/VOTRE-PROJECT-ID/flask-repo/flask-app
```

**Question 2 (compte-rendu) :** Pourquoi tagger une image avec plusieurs tags (SHA complet, SHA court, latest) ? Dans quel cas utilise-t-on chacun ?

## Partie 2 — Versioning sémantique (SemVer)

**Objectif :** Appliquer le versioning sémantique et créer des tags Git de version.

### 2.1 — Créer votre première version

Votre projet est fonctionnel et déployé. C'est le moment de créer la version 1.0.0 :

```
# Créer un tag annoté  
git tag -a v1.0.0 -m "Version 1.0.0 - première release stable"  
  
# Pousser le tag  
git push origin v1.0.0  
  
# Lister les tags  
git tag -l
```

### 2.2 — Simuler des évolutions

Ajoutez une fonctionnalité (MINOR) :

```
# Ajoutez une route /version dans src/app.py  
@app.route("/version")
```

```
def version():
    return jsonify({"version": "1.1.0"})

# Ajoutez le test dans tests/test_app.py
def test_version(client):
    response = client.get("/version")
    assert response.status_code == 200

git add .
git commit -m "feat: ajout route /version"
git push origin main
```

Puis corrigez un bug (PATCH) — le test ne vérifie pas le contenu :

```
# Améliorez le test dans tests/test_app.py
def test_version(client):
    response = client.get("/version")
    assert response.status_code == 200
    assert "version" in response.get_json()

git add .
git commit -m "fix: amélioration test route /version"
git push origin main
```

**Question 3 (compte-rendu) :** Expliquez le versioning sémantique (SemVer). Pour chaque cas, indiquez si c'est un changement MAJOR, MINOR ou PATCH : ajout d'une route, correction d'un bug, changement du format de réponse JSON.

**Question 4 (compte-rendu) :** Quelle est la différence entre un tag Git léger (`git tag v1.0`) et un tag annoté (`git tag -a v1.0 -m "..."`) ?

## Partie 3 — Releases GitHub automatisées

**Objectif :** Créer des releases GitHub automatiquement à partir des tags.

### 3.1 — Release manuelle (pour comprendre)

Sur GitHub, allez dans votre dépôt → Releases → Create a new release :

- Choose a tag : sélectionnez **v1.0.0**
- Release title : **v1.0.0 — Première release**
- Description : résumez les fonctionnalités
- Cliquez **Publish release**

Observez la page Releases de votre dépôt.

### 3.2 — Préparer le fichier de version

release-please a besoin d'un fichier de version pour le mettre à jour automatiquement. Ajoutez la version dans `pyproject.toml` :

```
# Ajoutez cette section dans pyproject.toml
[project]
name = "mon-projet-flask"
version = "1.0.0"

git add pyproject.toml
git commit -m "chore: ajout version dans pyproject.toml"
git push origin main
```

**Astuce :** release-please mettra automatiquement à jour ce numéro de version et créera un fichier CHANGELOG.md avec l'historique des changements.

### 3.3 — Automatiser avec release-please

release-please (Google) analyse vos Conventional Commits et crée automatiquement des releases avec un changelog. Créez le fichier `.github/workflows/release.yml` :

```
name: Release

on:
  push:
    branches: [main]

permissions:
  contents: write
  pull-requests: write

jobs:
  release:
    runs-on: ubuntu-latest
    outputs:
      release_created: ${ steps.release.outputs.release_created }
      tag_name: ${ steps.release.outputs.tag_name }
    steps:
      - uses: googleapis/release-please-action@v4
        id: release
        with:
          release-type: python

git add .github/workflows/release.yml
git commit -m "ci: ajout release-please pour releases automatiques"
git push origin main
```

### 3.4 — Observer le fonctionnement

Après le push, release-please va créer une PR intitulée "chore(main): release X.Y.Z". Cette PR contient le bump de version et le changelog généré. Quand vous la mergez, la release est créée automatiquement.

Faites un commit feat: et observez la PR créée :

```
# Ajoutez une petite fonctionnalité
git commit --allow-empty -m "feat: amélioration du health check"
git push origin main

# Allez sur GitHub → Pull Requests
```

```
# Vous devriez voir une PR de release-please  
# Mergez-la pour créer la release
```

**Question 5 (compte-rendu) : Comment release-please détermine-t-il le numéro de version à partir des commits ? Quel est le lien avec les Conventional Commits ?**

**Question 6 (compte-rendu) : Quel est l'avantage d'automatiser les releases plutôt que de les créer manuellement ?**

## Partie 4 — Pipeline de release complet

**Objectif :** Lier la release au build et déploiement d'une image Docker versionnée.

### 4.1 — Retirer le deploy de ci.yml

Au TP5, votre pipeline `ci.yml` contenait un job `deploy` qui se déclenchait à chaque push sur main. Maintenant que les déploiements sont gérés par `release.yml` (via les releases versionnées), **retirez le job `deploy` de `ci.yml`**.

Votre `ci.yml` ne doit garder que le job de tests/qualité/sécurité. Le déploiement est désormais déclenché uniquement par une release.

```
# Supprimez le job "deploy" de ci.yml  
# Ne gardez que le job "security-and-quality"  
git add .github/workflows/ci.yml  
git commit -m "ci: déplacement du deploy vers release.yml"  
git push origin main
```

**Attention :** Si vous gardez le `deploy` dans les deux workflows, l'application sera déployée deux fois à chaque release : une fois par `ci.yml` (push sur main) et une fois par `release.yml`.

### 4.2 — Déclencher le deploy sur une release

Modifiez votre workflow `release.yml` pour ajouter un job de build et deploy qui se déclenche uniquement quand une release est créée :

```
deploy-release:  
  needs: release  
  if: ${ needs.release.outputs.release_created }  
  runs-on: ubuntu-latest  
  steps:  
    - uses: actions/checkout@v4  
  
    - name: Authentification GCP  
      uses: google-github-actions/auth@v2  
      with:  
        credentials_json: ${ secrets.GCP_SA_KEY }  
  
    - name: Configurer Docker  
      run: gcloud auth configure-docker europe-west1-docker.pkg.dev
```

```
- name: Build et push avec tag de version
  run: |
    IMAGE=europe-west1-docker.pkg.dev/${{ secrets.GCP_PROJECT_ID }}/flask-re
po/flask-app
    VERSION=${{ needs.release.outputs.tag_name }}
    docker build -t $IMAGE:$VERSION -t $IMAGE:latest .
    docker push $IMAGE:$VERSION
    docker push $IMAGE:latest

- name: Déployer sur Cloud Run
  uses: google-github-actions/deploy-cloudrun@v2
  with:
    service: flask-app
    region: europe-west1
    image: europe-west1-docker.pkg.dev/${{ secrets.GCP_PROJECT_ID }}/flask-r
epo/flask-app:${{ needs.release.outputs.tag_name }}

git add .github/workflows/release.yml
git commit -m "ci: deploy sur release avec tag de version"
git push origin main
```

**Astuce :** Avec cette configuration, chaque release taggée (v1.1.0, v1.2.0...) déclenche un déploiement avec une image Docker taggée du numéro de version. Le rollback consiste à redéployer l'image d'une version précédente.

## 4.3 — Vérifier le pipeline complet

Testez le pipeline de bout en bout :

- Faites un commit feat: ajout fonctionnalité X
- Poussez sur main → la CI tourne
- release-please crée une PR de release
- Mergez la PR → la release est créée
- Le job deploy-release build l'image avec le tag de version
- L'image est déployée sur Cloud Run

Vérifiez la page Releases et l'image sur Artifact Registry.

**Question 7 (compte-rendu) :** Décrivez le pipeline de release complet, du commit au déploiement. Combien de workflows sont impliqués et quel est leur rôle ?

**Question 8 (compte-rendu) :** Quelle est la différence entre déployer avec `github.sha` (TP5) et déployer avec un tag de version SemVer (ce TP) ? Quand utiliser chacun ?

**Question 9 (compte-rendu) :** Pourquoi le principe d'immuabilité des artefacts est-il important ? Que se passe-t-il si on écrase un tag Docker existant ?

## Partie 5 — Recherche autonome

**Objectif :** Explorer les pratiques de versioning et de release en conditions réelles.

### 5.1 — Analyser les releases d'un projet open source

Choisissez un projet open source populaire sur GitHub (par exemple Flask, FastAPI, pytest, ou un autre projet que vous utilisez). Allez dans l'onglet **Releases** du projet.

**Question 10 (compte-rendu) :** Analysez les 5 dernières releases du projet choisi : quels types de changements (MAJOR, MINOR, PATCH) ? Le changelog est-il automatisé ou manuel ? Les tags suivent-ils SemVer ? Quels outils le projet utilise-t-il pour ses releases ? Indiquez le lien vers la page Releases du projet.

### 5.2 — Configurer une politique de rétention

Consultez la documentation Artifact Registry sur les politiques de nettoyage : <https://cloud.google.com/artifact-registry/docs/repositories/cleanup-policy>

Configurez une politique qui supprime automatiquement les images non taggées de plus de 30 jours dans votre dépôt Artifact Registry.

**Astuce :** En entreprise, sans politique de rétention, le registre grossit indéfiniment et les coûts de stockage explosent.



## Récapitulatif

| Concept              | Description                                       |
|----------------------|---------------------------------------------------|
| Artefact             | Livable immutable du pipeline (image, package...) |
| SemVer               | MAJOR.MINOR.PATCH (ex: v1.2.3)                    |
| Tag Git annoté       | git tag -a v1.0.0 -m "message"                    |
| Conventional Commits | feat:, fix:, docs: → versioning auto              |
| release-please       | Automatise releases + changelog                   |
| Multi-tagging        | SHA + SHA court + version + latest                |
| Immutabilité         | Ne jamais écraser un tag existant                 |
| GitHub Release       | Publication versionnée avec changelog             |

## Checklist de rendu

Avant de rendre, vérifiez que vous avez :

- Le multi-tagging configuré dans le pipeline (SHA + SHA court + latest)
- Au moins un tag Git de version créé (v1.0.0)
- release-please configuré et fonctionnel
- Au moins une release GitHub créée automatiquement avec changelog
- Un pipeline de release qui build et déploie avec le tag de version
- Un compte-rendu avec les réponses aux 10 questions